SDLC Area	Sub-Area / Process Area	Capability / Practice
Requirements	Product Discovery & Ideation	Idea generation & exploration
Requirements	Product Discovery & Ideation	Problem discovery & framing
Requirements	Requirement Engineering	Requirements elicitation and analysis
Requirements	Requirement Engineering	Technical feasibility and risk assessment
Requirements	Requirement Prioritization & Road-mapping	Dependency & sequencing analysis
Requirements	Requirement Prioritization & Road-mapping	Requirement prioritization & trade-off analysis
Requirements	Requirement Prioritization & Road-mapping	Roadmap scenario planning
Requirements	Requirements	Backlog Refinement
Requirements	Requirements	Bidirectional Traceability
Requirements	Requirements	Effort, Cost & Risk Simulation
Requirements	Requirements	Impact Analysis
Requirements	Requirements	OCM & Course Correction
Requirements	Requirements	Root-to-Live Discovery & Sequencing

Requirements	Requirements	Transcript → Requirements
Requirements	User Experience & Insights	Experience pain-point identification
Requirements	User Experience & Insights	Feedback & sentiment analysis
Requirements	User Experience & Insights	Persona & journey modeling
Requirements	User Experience & Insights	User research synthesis
Architecture	Architecture	Architecture Synthesiser
Architecture	Architecture	Auto-Generate Architecture Diagrams
Architecture	Architecture	Automated Drift Remediation
Architecture	Architecture	Compliance & Security Policy Advisor
Architecture	Architecture	Drift Detection in PRs
Architecture	Architecture	Extract Architecture from Code

Architecture	Architecture	FinOps Modeller & Planner
Architecture	Architecture & Design Governance	Architecture & Design Quality Governance
Architecture	Architecture & Design Governance	Architecture Decision Governance & Traceability
Architecture	Architecture & Design Governance	Architecture Decision Knowledge management
Architecture	Architecture & Design Governance	Architecture Decision Review & Approval Governance
Architecture	Architecture & Design Governance	Architecture and design reviews
Architecture	Architecture & Design Governance	Candidate Architecture Generation
Architecture	Architecture & Design Governance	Decision traceability
Architecture	Architecture & Design Governance	Design Documentation Governance & Traceability (or) Design Traceability Management
Architecture	Legacy Modernization & Decomposition	System Decomposition Analysis
Development	Build Process Management	Build Artifact generation and storage
Development	Build Process Management	Build Automation Engineering
Development	Build Process Management	Build Execution
Development	Build Process Management	Build Failure Management
Development	Build Process Management	Build performance optimization
Development	Code Development & Review	Code Branching & Merge Governance

Development	Code Development & Review	Code Generation
Development	Code Development & Review	Code Quality Standards Governance & Enforcement
Development	Code Development & Review	Code Review
Development	Code Development & Review	Code base Management
Development	Code Development & Review	Code refactoring and optimization
Development	Code Development & Review	Defect prevention and resolution
Development	Code Development & Review	Security Vulnerability
Development	Configuration & Dependency Management	Configuration file management
Development	Configuration & Dependency Management	Dependency management
Development	Configuration & Dependency Management	Source & Version Management Governance
Development	Development	AI Coding Assistant - Agent Mode
Development	Development	AI-Assisted Code Review
Development	Development	Custom Agents via MCP & Scripts

Development	Development	Custom Instructions & Skills
Development	Development	Long-Running Agentic Tasks
Development	Development	Orchestrator & Sub-Agent Architecture
Development	Technical Debt & Refactoring	Code Quality Measurement & Governance
Development	Technical Debt & Refactoring	Legacy code modernization
Development	Technical Debt & Refactoring	Refactoring Governance & Prioritization
Development	Technical Debt & Refactoring	Technical debt identification
Development	Unit & Component Testing	Unit Test coverage
Development	Unit & Component Testing	Unit test execution
Development	Unit & Component Testing	Unit test generation
Testing	Defect Management	Bug reproducibility insights
Testing	Defect Management	Defect Classification
Testing	Defect Management	Defect Triaging
Testing	Defect Management	Defect prioritization
Testing	Defect Management	Defect reporting

Testing	Defect Management	Defect resolution process
Testing	Defect Management	Defect tracking
Testing	Defect Management	Executive summary generation
Testing	Test Automation Governance	CI integration for test automation
Testing	Test Automation Governance	Test Automation Enablement & Governance
Testing	Test Automation Governance	Test Script Lifecycle Management
Testing	Test Automation Governance	Test automation strategy
Testing	Test Design & Development	Test Automation Architecture & Enablement
Testing	Test Design & Development	Test Coverage
Testing	Test Design & Development	Test Generation
Testing	Test Design & Development	Test Prioritization
Testing	Test Design & Development	Test case development and traceability
Testing	Test Design & Development	Test data management
Testing	Test Execution & Reporting	Defect logging and triage
Testing	Test Execution & Reporting	Flaky test detection
Testing	Test Execution & Reporting	Test Environment Scaling
Testing	Test Execution & Reporting	Test Execution Reporting & Quality Insights
Testing	Test Execution & Reporting	Test Quality Evidence & Insights Management
Testing	Test Execution & Reporting	Test Scripting
Testing	Test Execution & Reporting	Test Suite Optimization
Testing	Test Execution & Reporting	Test environment management

Testing	Test Execution & Reporting	Test execution planning & Orchestration
Testing	Test Strategy & Planning	Define test objectives and scope
Testing	Test Strategy & Planning	Identify test types and levels
Testing	Test Strategy & Planning	Resource and environment planning
Testing	Test Strategy & Planning	Risk-based test prioritization
Testing	Testing	E2E Test Execution Workflow
Testing	Testing	Non-Functional & Performance Testing
Testing	Testing	Offline Evaluation of AI Features
Testing	Testing	Synthetic Test Data Creation
Testing	Testing	Test Case Generation
Testing	Testing	Test Script Generation

Testing	Testing	Vulnerability Simulation
Deployment	CD / CI-CD	Automated Release Notes
Deployment	CD / CI-CD	Capacity Prediction
Deployment	CD / CI-CD	Deployment Script Generation
Deployment	CD / CI-CD	Initial Triage Assistance
Deployment	CD / CI-CD	Root Cause Analysis (RCA)
Deployment	CD / CI-CD	Runtime FinOps Optimisation
Deployment	CD / CI-CD	Self-Healing Systems
Deployment	CI / CI-CD	AI-Integrated Quality Gates

Deployment	CI / CI-CD	Agents Integrated in CI via API
Deployment	CI / CI-CD	Code Quality, AI Slop & Data Lineage Reporting
Deployment	CI / CI-CD	OSS, Deprecated Code & Evergreening Scans
Deployment	CI / CI-CD	Pipeline Creation Using AI
Deployment	CI / CI-CD	Shift-Left Agents in Developer IDE
Deployment	Continuous Integration & Delivery	CI/CD Flow Optimization & Throughput Engineering
Deployment	Continuous Integration & Delivery	Continuous Integration & Deployment Automation
Deployment	Continuous Integration & Delivery	Pipeline Reliability Forecasting & Prevention
Deployment	Continuous Integration & Delivery	Release Risk Assessment & Controls
Deployment	Continuous Integration & Delivery	Release Rollback & Recovery Management
Deployment	Continuous Integration & Delivery	Release gating and approvals

Questionnaire
To what extent does your team use AI for idea generation & exploration?
To what extent does your team use AI for problem discovery & framing?
To what extent does your team use AI for requirements elicitation and analysis?
To what extent does your team use AI for technical feasibility and risk assessment?
To what extent does your team use AI for dependency & sequencing analysis?
To what extent does your team use AI for requirement prioritization & trade-off analysis?
To what extent does your team use AI for roadmap scenario planning?
To what extent is the following true in your current practice: AI decomposes vague backlog items into sized features with acceptance criteria. The engineer's role is to judge and adjust AI output, not to write stories from a blank page.
To what extent is the following true in your current practice: The agent maintains and enforces traceability automatically on every PR. The engineer sets the traceability policy once; the agent applies it continuously without prompting.
To what extent is the following true in your current practice: The agent monitors cost and risk continuously against thresholds the team has set, and alerts automatically when they are breached. Humans are not periodically reviewing a spreadsheet - the system comes to them.
To what extent is the following true in your current practice: An agent independently traces a change request across the codebase and produces an impact matrix. The engineer validates the output - they do not do the tracing themselves.
To what extent is the following true in your current practice: The system catches scope or risk drift and raises it before a human notices. If a human is always the one spotting drift, this practice is not Achieved.
To what extent is the following true in your current practice: When scope changes, the system replans automatically and escalates only when sequencing decisions exceed defined risk thresholds. A human does not replan by hand - they approve or override the agent's plan.

To what extent is the following true in your current practice: The first draft of every requirement comes from AI processing meeting notes or transcripts - not from a human writing from scratch. The engineer directs, refines, and approves; authoring is AI's job.
To what extent does your team use AI for experience pain-point identification?
To what extent does your team use AI for feedback & sentiment analysis?
To what extent does your team use AI for persona & journey modeling?
To what extent does your team use AI for user research synthesis?
To what extent is the following true in your current practice: An agent produces target-state blueprint options with trade-off analysis. The architect selects and approves a direction - they do not synthesise options themselves. The agent does the synthesis work.
To what extent is the following true in your current practice: The first version of any architecture diagram is AI-generated from code, IaC, or natural language. An engineer directing AI to produce a diagram, then refining it, is the standard workflow - not an engineer drawing from scratch.
To what extent is the following true in your current practice: Low-risk drift is remediated and shipped automatically when tests pass and the change meets defined guardrails. Humans are only pulled in when the drift is high-risk or the guardrails cannot classify it. Humans set the guardrails - the agent enforces them.
To what extent is the following true in your current practice: The system notices a compliance gap before a person does and raises it automatically. If a human is the one discovering policy violations, the monitoring is not at this level.
To what extent is the following true in your current practice: The agent inspects every PR for architecture drift and can block a clear violation without a human stepping in. If a human must read every PR to catch drift, the gate is not real.
To what extent is the following true in your current practice: AI reads the codebase and generates architecture documentation and component maps. Engineers do not draw diagrams from memory - they review and correct AI-generated output. Critically, the AI can see the actual codebase, not just generic knowledge.

To what extent is the following true in your current practice: The agent flags architecture cost implications and overspend automatically against thresholds the team has set. Cost decisions are not discovered in a monthly review - the system surfaces them in real time.
To what extent does your team use AI for architecture & design quality governance?
To what extent does your team use AI for architecture decision governance & traceability?
To what extent does your team use AI for architecture decision knowledge management?
To what extent does your team use AI for architecture decision review & approval governance?
To what extent does your team use AI for architecture and design reviews?
To what extent does your team use AI for candidate architecture generation?
To what extent does your team use AI for decision traceability?
To what extent does your team use AI for design documentation governance & traceability (or) design traceability management?
To what extent does your team use AI for system decomposition analysis?
To what extent does your team use AI for build artifact generation and storage?
To what extent does your team use AI for build automation engineering?
To what extent does your team use AI for build execution?
To what extent does your team use AI for build failure management?
To what extent does your team use AI for build performance optimization?
To what extent does your team use AI for code branching & merge governance?

To what extent does your team use AI for code generation?
To what extent does your team use AI for code quality standards governance & enforcement?
To what extent does your team use AI for code review?
To what extent does your team use AI for code base management?
To what extent does your team use AI for code refactoring and optimization?
To what extent does your team use AI for defect prevention and resolution?
To what extent does your team use AI for security vulnerability?
To what extent does your team use AI for configuration file management?
To what extent does your team use AI for dependency management?
To what extent does your team use AI for source & version management governance?
To what extent is the following true in your current practice: Every engineer uses AI in agent mode for multi-step tasks - not just autocomplete. If removing AI tools would only affect speed and leave the process unchanged, this is the level the team is at.
To what extent is the following true in your current practice: AI participates in every code review, flagging bugs and security issues. A human reads every flag and makes the final call - the AI accelerates the reviewer, it does not replace the human gate.
To what extent is the following true in your current practice: An agent can open a pull request on its own without a human writing the change by hand. The human's role is to review and approve the PR - not to author the code. This is the line between L1 and L2.

To what extent is the following true in your current practice: AI can see the team's own codebase, standards, and project context - not just generic public knowledge. Custom instructions or skills are in place so the agent is project-aware by default, not by the engineer pasting context manually each time.
To what extent is the following true in your current practice: Agents operate autonomously over extended periods. The safety net - automated rollback, fast catch-and-undo - is what makes this level achievable. If a wrong agent action cannot be caught and reversed quickly, the team cannot safely operate here.
To what extent is the following true in your current practice: Complex tasks are handled by orchestrated multi-agent systems that plug into a shared internal platform - not individual engineers wiring up their own tools. Humans set the guardrails and escalation rules; agents handle execution and peer review within those rules.
To what extent does your team use AI for code quality measurement & governance?
To what extent does your team use AI for legacy code modernization?
To what extent does your team use AI for refactoring governance & prioritization?
To what extent does your team use AI for technical debt identification?
To what extent does your team use AI for unit test coverage?
To what extent does your team use AI for unit test execution?
To what extent does your team use AI for unit test generation?
To what extent does your team use AI for bug reproducibility insights?
To what extent does your team use AI for defect classification?
To what extent does your team use AI for defect triaging?
To what extent does your team use AI for defect prioritization?
To what extent does your team use AI for defect reporting?

To what extent does your team use AI for defect resolution process?
To what extent does your team use AI for defect tracking?
To what extent does your team use AI for executive summary generation?
To what extent does your team use AI for ci integration for test automation?
To what extent does your team use AI for test automation enablement & governance?
To what extent does your team use AI for test script lifecycle management?
To what extent does your team use AI for test automation strategy?
To what extent does your team use AI for test automation architecture & enablement?
To what extent does your team use AI for test coverage?
To what extent does your team use AI for test generation?
To what extent does your team use AI for test prioritization?
To what extent does your team use AI for test case development and traceability?
To what extent does your team use AI for test data management?
To what extent does your team use AI for defect logging and triage?
To what extent does your team use AI for flaky test detection?
To what extent does your team use AI for test environment scaling?
To what extent does your team use AI for test execution reporting & quality insights?
To what extent does your team use AI for test quality evidence & insights management?
To what extent does your team use AI for test scripting?
To what extent does your team use AI for test suite optimization?
To what extent does your team use AI for test environment management?

To what extent does your team use AI for test execution planning & orchestration?
To what extent does your team use AI for define test objectives and scope?
To what extent does your team use AI for identify test types and levels?
To what extent does your team use AI for resource and environment planning?
To what extent does your team use AI for risk-based test prioritization?
To what extent is the following true in your current practice: The system can block a bad change without a human clicking approve. Agents generate, run, and triage failures automatically. If a human must approve every test run result, the quality gate is not real.
To what extent is the following true in your current practice: SLA thresholds are set by the team; the agent enforces them on every change. Non-functional regressions are caught automatically - a human does not spot them in a periodic performance review.
To what extent is the following true in your current practice: For any feature with an AI component, the agent runs structured evals against a golden dataset before code ships. The human sets the golden dataset and pass threshold once; the agent enforces it on every change.
To what extent is the following true in your current practice: The agent generates test data at scale from schema definitions and edge-case rules set by the engineer. A human does not curate or maintain test datasets manually - the agent produces them on demand.
To what extent is the following true in your current practice: AI generates test cases from requirements covering happy paths, edge cases, and negative scenarios. A human reviews and selects which cases to use - the engineer does not write test cases from scratch.
To what extent is the following true in your current practice: Tests run automatically on every change - triggered by the pipeline, not by a human. The agent generates and executes scripts; a human reviews results and decides whether to proceed to the next stage.

To what extent is the following true in your current practice: Adversarial probes and prompt injection tests run as part of every standard pipeline. Security regressions are caught by the system before anyone on the team notices - not by periodic manual pen testing.
To what extent is the following true in your current practice: An agent generates release notes from commit history and PRs. The engineer approves before distribution - they do not write release notes by hand.
To what extent is the following true in your current practice: The system predicts capacity needs and triggers scaling recommendations before engineers react to a problem. If scaling decisions are made reactively by a human watching metrics, this is not Achieved.
To what extent is the following true in your current practice: AI generates deployment scripts, Helm charts, and YAML manifests. The engineer reviews and triggers deployment - they do not write deployment config from scratch.
To what extent is the following true in your current practice: When something breaks in production, a system notices and surfaces probable causes before an engineer manually investigates. If an engineer is always the first to notice an incident, this practice is not Achieved.
To what extent is the following true in your current practice: When something breaks, a system triages it and produces a structured RCA before a human investigates. The engineer validates and acts on the analysis - they do not start the investigation from raw logs.
To what extent is the following true in your current practice: The agent rightsizes resources and terminates idle instances automatically within cost thresholds the team has set. Cost is not managed by a human reviewing spend reports - the system optimises continuously.
To what extent is the following true in your current practice: When a service degrades, the system catches it, executes remediation, and - if the fix meets the defined guardrails - resolves it before anyone on the team knows it happened. Humans are pulled in only for ambiguous or high-risk incidents.
To what extent is the following true in your current practice: AI quality checks - linting, security scanning, coverage - run automatically on every commit. A human reads the results and decides whether to proceed. The checks run themselves; the human is still the gate.

To what extent is the following true in your current practice: AI agents run as first-class CI pipeline steps via API, performing analysis and validation as part of the automated build. The agent acts in the pipeline; a human reviews output before merge.
To what extent is the following true in your current practice: The system blocks quality regressions and low-quality AI-generated code automatically on every pipeline run. A human does not periodically audit for code quality - the gate is always on.
To what extent is the following true in your current practice: Low-risk dependency updates and evergreening PRs ship automatically within guardrails the team has set. A human does not review every maintenance PR - the system handles routine updates and escalates only non-routine ones.
To what extent is the following true in your current practice: An agent generates CI pipeline configuration from requirements. The engineer reviews and approves - they do not write YAML or pipeline scripts from scratch.
To what extent is the following true in your current practice: The same agents that run in CI are available in the developer's IDE via a shared platform (MCP), not individually wired up by each engineer. Issues are caught before commit using the same rules as CI.
To what extent does your team use AI for ci/cd flow optimization & throughput engineering?
To what extent does your team use AI for continuous integration & deployment automation?
To what extent does your team use AI for pipeline reliability forecasting & prevention?
To what extent does your team use AI for release risk assessment & controls?
To what extent does your team use AI for release rollback & recovery management?
To what extent does your team use AI for release gating and approvals?

Guidance / Definition	Level
Ability to use AI to generate, cluster, and explore solution ideas based on problems, constraints, and prior learnings.	
Ability to use AI to analyze user signals, business data, and market inputs to identify and frame high-value problems to solve.	
Ability to use AI to capture, synthesize, and structure requirements from conversations and documents into actionable, traceable specifications.	
Ability to use AI to assess technical feasibility, constraints, dependencies, and risks early and produce decision-ready insights with explainable rationale.	
Ability to use AI to identify requirement dependencies and recommend optimal sequencing.	
Ability to use AI to prioritize requirements using value, cost, risk, dependency, and capacity signals.	
Ability to use AI to generate and compare roadmap scenarios under different constraints and assumptions.	
AI decomposes vague backlog items into sized features with acceptance criteria. The engineer's role is to judge and adjust AI output, not to write stories from a blank page.	
The agent maintains and enforces traceability automatically on every PR. The engineer sets the traceability policy once; the agent applies it continuously without prompting.	
The agent monitors cost and risk continuously against thresholds the team has set, and alerts automatically when they are breached. Humans are not periodically reviewing a spreadsheet - the system comes to them.	
An agent independently traces a change request across the codebase and produces an impact matrix. The engineer validates the output - they do not do the tracing themselves.	
The system catches scope or risk drift and raises it before a human notices. If a human is always the one spotting drift, this practice is not Achieved.	
When scope changes, the system replans automatically and escalates only when sequencing decisions exceed defined risk thresholds. A human does not replan by hand - they approve or override the agent's plan.	

The first draft of every requirement comes from AI processing meeting notes or transcripts - not from a human writing from scratch. The engineer directs, refines, and approves; authoring is AI's job.	
Ability to use AI to detect UX pain points across journeys using qualitative and quantitative signals.	
Ability to use AI to analyze user feedback and sentiment at scale to inform requirements and prioritization.	
Ability to use AI to generate and evolve user personas and journeys based on behavioral and contextual data.	
Ability to use AI to synthesize user research inputs (interviews, surveys, analytics) into actionable insights and themes.	
An agent produces target-state blueprint options with trade-off analysis. The architect selects and approves a direction - they do not synthesise options themselves. The agent does the synthesis work.	
The first version of any architecture diagram is AI-generated from code, IaC, or natural language. An engineer directing AI to produce a diagram, then refining it, is the standard workflow - not an engineer drawing from scratch.	
Low-risk drift is remediated and shipped automatically when tests pass and the change meets defined guardrails. Humans are only pulled in when the drift is high-risk or the guardrails cannot classify it. Humans set the guardrails - the agent enforces them.	
The system notices a compliance gap before a person does and raises it automatically. If a human is the one discovering policy violations, the monitoring is not at this level.	
The agent inspects every PR for architecture drift and can block a clear violation without a human stepping in. If a human must read every PR to catch drift, the gate is not real.	
AI reads the codebase and generates architecture documentation and component maps. Engineers do not draw diagrams from memory - they review and correct AI-generated output. Critically, the AI can see the actual codebase, not just generic knowledge.	

The agent flags architecture cost implications and overspend automatically against thresholds the team has set. Cost decisions are not discovered in a monthly review - the system surfaces them in real time.	
Ability to use AI to automate, assist, or augment 'Architecture & Design Quality Governance' with appropriate guardrails and measurable outcomes.	
Ability to use AI to capture and govern architecture decisions, surface trade-offs, and maintain traceability across requirements, designs, and implementation.	
Ability to use AI to curate and retrieve architecture decision knowledge (e.g., ADRs) to accelerate reuse, consistency, and decision quality across teams.	
Ability to use AI to review architectures and designs against standards and quality attributes, flag issues, and recommend improvements with evidence.	
Ability to use AI to review architectures and designs against standards and quality attributes, flag issues, and recommend improvements with evidence.	
The creative process of conceptualizing and outlining multiple potential high-level system architecture patterns that satisfy a given set of functional and non-functional requirements.	
Ability to use AI to automatically link decisions to drivers, requirements, designs, code, and outcomes and detect missing or broken links.	
Ability to use AI to generate and maintain design documentation and automatically keep it aligned and traceable to requirements, decisions, and code changes.	
The analytical process of evaluating an existing application's codebase and functionality to identify logical boundaries for refactoring into smaller, independent services or modules.	
Ability to use AI to validate artefact provenance, detect anomalies, and optimize artefact storage/retention for traceability and cost.	
Ability to use AI to design, generate, and maintain automated build pipelines/scripts with governance and reliability guardrails.	
Ability to use AI to optimize build execution (caching/parallelism), predict failures, and provide intelligent build diagnostics.	
Ability to use AI to classify build failures, isolate root causes, recommend fixes, and automate remediation steps where safe.	
Ability to use AI to propose and apply refactoring/performance improvements validated through tests, benchmarks, and policy constraints.	
Ability to use AI to enforce branching/PR policies, detect risky merges, and recommend safe merge strategies using change-impact context.	

Ability to use AI to generate production-grade code from intent/specs with security, quality, and traceability guardrails.	
Ability to use AI to continuously check and enforce coding standards/quality gates, explain violations, and suggest safe auto-fixes.	
Ability to use AI to auto-review code changes for defects, security, style, and architecture alignment and produce actionable feedback with risk signals.	
Ability to use AI to understand codebase structure, detect duplication/anti-patterns, and recommend modularization and maintainability improvements.	
Ability to use AI to propose and apply refactoring/performance improvements validated through tests, benchmarks, and policy constraints.	
Ability to use AI to predict defect-prone changes, pinpoint likely root causes, recommend fixes, and prevent recurrence via learned patterns.	
Ability to use AI to identify, prioritize, and remediate code-level vulnerabilities and insecure patterns with exploitability context.	
Ability to use AI to generate/validate configuration, detect misconfigurations, and enforce consistency across environments and policies.	
Ability to use AI to recommend safe dependency upgrades, detect vulnerable/unused dependencies, and forecast build/runtime/licensing impact.	
Ability to use AI to enforce source-control/versioning standards, detect policy drift, and automate compliance reporting and corrective actions.	
Every engineer uses AI in agent mode for multi-step tasks - not just autocomplete. If removing AI tools would only affect speed and leave the process unchanged, this is the level the team is at.	
AI participates in every code review, flagging bugs and security issues. A human reads every flag and makes the final call - the AI accelerates the reviewer, it does not replace the human gate.	
An agent can open a pull request on its own without a human writing the change by hand. The human's role is to review and approve the PR - not to author the code. This is the line between L1 and L2.	

AI can see the team's own codebase, standards, and project context - not just generic public knowledge. Custom instructions or skills are in place so the agent is project-aware by default, not by the engineer pasting context manually each time.	
Agents operate autonomously over extended periods. The safety net - automated rollback, fast catch-and-undo - is what makes this level achievable. If a wrong agent action cannot be caught and reversed quickly, the team cannot safely operate here.	
Complex tasks are handled by orchestrated multi-agent systems that plug into a shared internal platform - not individual engineers wiring up their own tools. Humans set the guardrails and escalation rules; agents handle execution and peer review within those rules.	
Ability to use AI to interpret code-quality signals, identify systemic issues, and drive governance actions backed by trend insights.	
Ability to use AI to understand legacy code, propose modernization paths, assist migration/refactor, and validate equivalence using tests.	
Ability to use AI to propose and apply refactoring/performance improvements validated through tests, benchmarks, and policy constraints.	
Ability to use AI to detect technical-debt hotspots, classify debt types, and quantify impact on delivery speed, risk, and cost.	
Ability to use AI to analyze unit-test coverage gaps, recommend high-value tests, and optimize for risk-based coverage rather than raw percentages.	
Ability to use AI to orchestrate unit test runs, diagnose failures, detect flakiness, and suggest fixes with confidence indicators.	
Ability to use AI to generate meaningful unit tests (including edge cases and mocks) aligned to code intent and quality standards.	
Ability to use AI to infer reproducibility steps from telemetry/logs and suggest minimal reproductions.	
Ability to use AI to classify defects by type, component, severity, and likely cause patterns for better routing and analytics.	
Ability to use AI to route defects to the right teams, propose severity, and cluster duplicates with confidence scoring.	
Ability to use AI to prioritize defects using business impact, risk, customer signals, and fix effort predictions.	
Ability to use AI to generate clear defect reports with evidence, context, and recommended next steps.	

Ability to use AI to predict defect-prone changes, pinpoint likely root causes, recommend fixes, and prevent recurrence via learned patterns.	
Ability to use AI to track defect aging, predict spillovers, and recommend interventions to reduce backlog risk.	
Ability to use AI to generate leadership-ready summaries of quality posture, top risks, and trends from defect data.	
Ability to use AI to integrate and tune automated tests in CI with intelligent selection, gating, and failure diagnosis.	
Ability to use AI to enforce automation standards, suggest best practices, and monitor automation health and ROI.	
Ability to use AI to detect test-script drift from product changes and automate safe updates and refactors.	
Ability to use AI to recommend an automation strategy aligned to risk, ROI, and maintainability constraints.	
Ability to use AI to design and evolve test automation frameworks, patterns, and utilities for scalable automation.	
Ability to use AI to assess functional and risk coverage gaps and recommend high-value tests and data.	
Ability to use AI to generate automated/manual tests from user flows, APIs, and specs with maintainable structure.	
Ability to use AI to recommend which tests to run first based on change impact, flakiness, and risk.	
Ability to use AI to generate test cases and maintain traceability to requirements, risks, and defects.	
Ability to use AI to generate, mask, subset, and maintain test data aligned to scenarios and privacy constraints.	
Ability to use AI to auto-create defect tickets with repro steps, logs, and suspected root causes and route them for triage.	
Ability to use AI to identify flaky tests, infer causes, and recommend stabilization actions automatically.	
Ability to use AI to scale test environments dynamically based on demand and optimize cost/performance.	
Ability to use AI to summarize test execution outcomes, detect quality trends, and generate decision-ready quality insights.	
Ability to use AI to curate quality evidence (logs, traces, screenshots), derive insights, and maintain auditability.	
Ability to use AI to generate and maintain test scripts (UI/API) with resilient locators and reusable components.	
Ability to use AI to propose and apply refactoring/performance improvements validated through tests, benchmarks, and policy constraints.	
Ability to use AI to manage test environment health and contention and recommend corrective actions.	

Ability to use AI to plan and orchestrate test runs across suites, environments, and pipelines for fastest feedback.	
Ability to use AI to derive test objectives and scope from requirements, risk signals, and change context with traceable justification.	
Ability to use AI to recommend test types and levels (unit/integration/e2e/non-functional) based on risk and architecture.	
Ability to use AI to forecast testing resources and environment needs and optimize schedules and capacity.	
Ability to use AI to prioritize testing using change impact, defect history, and business criticality to maximize risk coverage.	
The system can block a bad change without a human clicking approve. Agents generate, run, and triage failures automatically. If a human must approve every test run result, the quality gate is not real.	
SLA thresholds are set by the team; the agent enforces them on every change. Non-functional regressions are caught automatically - a human does not spot them in a periodic performance review.	
For any feature with an AI component, the agent runs structured evals against a golden dataset before code ships. The human sets the golden dataset and pass threshold once; the agent enforces it on every change.	
The agent generates test data at scale from schema definitions and edge-case rules set by the engineer. A human does not curate or maintain test datasets manually - the agent produces them on demand.	
AI generates test cases from requirements covering happy paths, edge cases, and negative scenarios. A human reviews and selects which cases to use - the engineer does not write test cases from scratch.	
Tests run automatically on every change - triggered by the pipeline, not by a human. The agent generates and executes scripts; a human reviews results and decides whether to proceed to the next stage.	

Adversarial probes and prompt injection tests run as part of every standard pipeline. Security regressions are caught by the system before anyone on the team notices - not by periodic manual pen testing.	
An agent generates release notes from commit history and PRs. The engineer approves before distribution - they do not write release notes by hand.	
The system predicts capacity needs and triggers scaling recommendations before engineers react to a problem. If scaling decisions are made reactively by a human watching metrics, this is not Achieved.	
AI generates deployment scripts, Helm charts, and YAML manifests. The engineer reviews and triggers deployment - they do not write deployment config from scratch.	
When something breaks in production, a system notices and surfaces probable causes before an engineer manually investigates. If an engineer is always the first to notice an incident, this practice is not Achieved.	
When something breaks, a system triages it and produces a structured RCA before a human investigates. The engineer validates and acts on the analysis - they do not start the investigation from raw logs.	
The agent rightsizes resources and terminates idle instances automatically within cost thresholds the team has set. Cost is not managed by a human reviewing spend reports - the system optimises continuously.	
When a service degrades, the system catches it, executes remediation, and - if the fix meets the defined guardrails - resolves it before anyone on the team knows it happened. Humans are pulled in only for ambiguous or high-risk incidents.	
AI quality checks - linting, security scanning, coverage - run automatically on every commit. A human reads the results and decides whether to proceed. The checks run themselves; the human is still the gate.	

AI agents run as first-class CI pipeline steps via API, performing analysis and validation as part of the automated build. The agent acts in the pipeline; a human reviews output before merge.	
The system blocks quality regressions and low-quality AI-generated code automatically on every pipeline run. A human does not periodically audit for code quality - the gate is always on.	
Low-risk dependency updates and evergreening PRs ship automatically within guardrails the team has set. A human does not review every maintenance PR - the system handles routine updates and escalates only non-routine ones.	
An agent generates CI pipeline configuration from requirements. The engineer reviews and approves - they do not write YAML or pipeline scripts from scratch.	
The same agents that run in CI are available in the developer's IDE via a shared platform (MCP), not individually wired up by each engineer. Issues are caught before commit using the same rules as CI.	
Ability to use AI to propose and apply refactoring/performance improvements validated through tests, benchmarks, and policy constraints.	
Ability to use AI to generate/maintain CI/CD automation and improve pipeline outcomes with guardrails and auditability.	
Ability to use AI to predict pipeline failures, recommend preventive fixes, and reduce recurrence through learned patterns.	
Ability to use AI to assess technical feasibility, constraints, dependencies, and risks early and produce decision-ready insights with explainable rationale.	
Ability to use AI to recommend rollback/recovery actions, validate readiness, and accelerate restoration with safe automation.	
Ability to use AI to evaluate release evidence, recommend gate decisions, and automate approvals within policy constraints.	

AI tools used	Remarks

